produces:

```
#<Proc:0x0000000104cb8c20 prog.rb:5>
nil
```

- By calling Proc.new, again associating it with a block. There's also a built-in Kernel#proc method, which is the same as Proc.new. The use of Kernel#proc is not recommended in new code.

```
block = Proc.new { "a block" }
block     # => #<Proc:0x0000000102979048 prog.rb:1>
```

- By calling the method Kernel#lambda and associating a block with the call.

```
block = lambda { "a block" }
block     # => #<Proc:0x00000001049190d8 prog.rb:1 (lambda)>
```

- Using the -> syntax.

```
lam = -> (p1, p2) { p1 + p2 }
lam.call(4, 3) # => 7
```

The first two styles of Proc object are identical in use. We'll call these objects *raw procs*. The third and fourth styles, generated by lambda and ->, add some functionality to the Proc object, as we'll see in a minute. We'll call these objects *lambdas*.

Here's the big thing to remember: raw procs are designed to work as the bodies of control structures such as loops. Lambdas are intended to act like methods. So, lambdas are stricter when checking the parameters passed to them, and a return in a lambda exits much as it would from a method.

Calling a Proc

You call a proc by invoking its methods call, yield, or []. The three forms are identical. Each takes arguments that are passed to the proc, just as if it were a regular method. If the proc is a lambda, Ruby will check that the number of supplied arguments match the expected parameters. You can also invoke a proc using the syntax name.(_args..._). This is mapped internally into name.call(args...).

Procs, break, and next

Within both raw procs and lambdas, executing next causes the block to exit back to the caller of the block. The return value is the value (or values) passed to next, or nil if no values are passed.

```
def ten_times
  10.times do |i|
    if yield(i)
      puts "Caller likes #{i}"
    end
  end
end

ten_times do |number|
next(true) if number ==7
end
```